# PLP - Recuperatorio del Primer Parcial - 1er cuatrimestre de 2024

| #Orden | Nro. Libreta | Apellido(s) | Nombre(s) |
|---|---|---|---|
| 81 | [redacted] | Fidlkowski | Valentín. |

| Corregido por | Nota E1 | Nota E2 | Nota E3 | Nota Final |
|---|---|---|---|---|
| PERLA | B- | B- | B- | A |

Este examen se aprueba obteniendo al menos dos ejercicios bien menos (B-) y uno regular (R). Las notas para cada ejercicio son: -, I, R, B-, B. Entregar cada ejercicio en hojas separadas. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras.
El orden de los ejercicios es arbitrario. Recomendamos leer el parcial completo antes de empezar a resolverlo.

## Ejercicio 1 - Programación funcional

**Aclaración: en este ejercicio no está permitido utilizar recursión explícita, a menos que se indique lo contrario.**

En este ejercicio vamos a modelar lógica proposicional en Haskell, de modo de poder construir fórmulas proposicionales y evaluarlas bajo distintas valuaciones.

```
data Prop = Var String | No Prop | Y Prop Prop | O Prop Prop | Imp Prop Prop

type Valuación = String -> Bool
```

Por ejemplo, la expresión: `Y (Var ''P'') (No (Imp (Var ''Q'') (Var ''R'')))` representa la proposición $P \wedge \neg(Q \Rightarrow R)$.

Las valuaciones se representan como funciones que a cada variable proposicional le asignan un valor booleano. Por ejemplo la valuación `\x -> x == ''P''` le asigna el valor verdadero a la variable $P$ y falso a todas las otras variables proposicionales.

a) Dar el tipo y definir las funciones `foldProp` y `recProp`, que implementan respectivamente los esquemas de recursión estructural y primitiva para el tipo `Prop`. Solo en este inciso se permite usar recursión explícita.

b) Definir la función `variables :: Prop -> [String]`, que dada una fórmula devuelve la lista con todas sus variables proposicionales en algún orden, sin elementos repetidos.

   Por ejemplo: `variables (O (Var ''P'') (No (Y (Var ''Q'') (Var ''P''))))` debería devolver la lista `[''P'',''Q'']` o la lista `[''Q'',''P'']`.

c) Definir la función `evaluar :: Valuación -> Prop -> Bool`, que indica si una fórmula es verdadera o falsa para una valuación dada.

d) Definir la función `estáEnFNN :: Prop -> Bool`, que indica si una fórmula está en Forma Normal Negada. Es decir, si no tiene implicaciones y la negación se aplica únicamente a variables y no a proposiciones más complejas.

   Por ejemplo: `Y (Var ''P'') (No (Imp (Var ''Q'') (Var ''R'')))` no está en FNN, y en cambio `Y (Var ''P'') (Y (Var ''Q'') (No (Var ''R'')))` sí lo está.

**Ejercicio 2 - Demostración e inferencia**

Considerar las siguientes definiciones sobre árboles con información en las hojas[1]:

```
data AIH a = Hoja a | Bin (AIH a) (AIH a)

     esHoja :: AIH a -> Bool
{E0} esHoja (Hoja x) = True
{E1} esHoja (Bin i d) = False

     izq :: AIH a -> AIH a
{I}  izq (Bin i d) = i

     der :: AIH a -> AIH a
{D}  der (Bin i d) = d

     mismaEstructura :: AIH a -> AIH a -> Bool
{M0} mismaEstructura (Hoja x) = esHoja
{M1} mismaEstructura (Bin i d) = \t -> not (esHoja t) &&
        mismaEstructura i (izq t) && mismaEstructura d (der t
```

a) Demostrar la siguiente propiedad:

$\forall$ `t::AIH a` . $\forall$ `u::AIH a` . mismaEstructura t u = mismaEstructura u t

Se recomienda hacer inducción en el primer árbol, utilizando extensionalidad en el segundo. Se permite definir macros (poner nombres a expresiones largas para no tener que repetirlas).

No es obligatorio reescribir los $\forall$ correspondientes en cada paso, pero es importante recordar que están presentes y escribir los que correspondan al platear la propiedad como predicado unario. Recordar también que los = de las definiciones pueden leerse en ambos sentidos.

Se consideran demostradas todas las propiedades conocidas sobre enteros y booleanos.

b) Usar el algoritmo **W** para inferir juicios de tipado válidos para las siguientes expresiones, o indicar por qué no es posible (recordar que en inferencia **no está permitido** renombrar variables):

i) $(\lambda x.x(\lambda x.\mathsf{Succ}(x)))(\lambda x.x)$

ii) $\lambda x.\mathsf{if}\ \mathsf{isZero}(x)\ \mathsf{then}\ x\ \mathsf{else}\ x\ \mathsf{zero}$

**Ejercicio 3 - Cálculo Lambda Tipado**

Se desea extender el cálculo lambda simplemente tipado para modelar **Árboles con información en las hojas**. Para eso se extienden los tipos y expresiones de la siguiente manera:

$\tau ::= \cdots \mid \mathtt{AIH}(\tau)$

$M ::= \cdots \mid \mathtt{Hoja}(M) \mid \mathtt{Bin}(M, M) \mid \mathsf{case}\ M\ \mathsf{of}\ \mathsf{Hoja}\ x \rightsquigarrow M;\ \mathsf{Bin}(i, d) \rightsquigarrow M$

- $\mathtt{AIH}(\tau)$ es el tipo de los árboles con información en las hojas de tipo $\tau$.
- $\mathtt{Hoja}(M)$ es un árbol compuesto por una única hoja con información $M$.
- $\mathtt{Bin}(M_1, M_2)$ es un árbol compuesto por dos subárboles $M_1$ y $M_2$.
- El observador $\mathsf{case}\ M_1\ \mathsf{of}\ \mathsf{Hoja}\ x \rightsquigarrow M_2;\ \mathsf{Bin}(i, d) \rightsquigarrow M_3$ permite acceder al valor de un árbol que es hoja (el cual se ligará a la variable $x$ que puede aparecer libre en $M_2$), y a los dos subárboles de un árbol que no es hoja (los cuales se ligarán a las variables $i$ y $d$ que pueden aparecer libres en $M_3$).

a. Introducir las reglas de tipado para la extensión propuesta.

b. Definir el conjunto de valores y las nuevas reglas de semántica operacional a pequeños pasos, tanto de congruencia como de cómputo.

c. Mostrar paso por paso cómo reduce la expresión:

$\mathsf{case}\ (\lambda n\colon \mathsf{Nat}.\ \mathtt{Hoja}(n))\ \mathsf{Succ}(\mathsf{zero})\ \mathsf{of}\ \mathsf{Hoja}\ x \rightsquigarrow \mathsf{Succ}(\mathsf{Pred}(x));\ \mathsf{Bin}(i, d) \rightsquigarrow \mathsf{zero}$

d. Definir como macro la función $\mathsf{esHoja}_\tau$, que toma un $\mathtt{AIH}(\tau)$ y devuelve un booleano que indica si es una hoja.

[1] Escritas con recursión explícita para facilitar las demostraciones.

1) a) foldProp :: (String → a) → (a → a) → (a → a → a) → (a → a → a)
→ (a → a → a) → Prop → a.

Grupo 1
Valentín Frankowski
Nro. = 81

~~foldProp fvar fnot fy fo fimp~~

foldProp fvar _ _ _ _ (Var s) = (fvar s)

foldProp fvar fnot fy fo fimp (Prop) = case of Prop

(No p) = fnot (recn p)

(Y p1 p2) = fy (recn p1) (recn p2)

(O p1 p2) = fo (recn p1) (recn p2)

(Imp p1 p2) = fimp (recn p1) (recn p2).

es Prop con P minúscula, no es el tipo de datos Prop (con P mayúscula).

where recn = foldProp fvar fnot fy fo fimp.

recProp :: (String → a) → (Prop → a → a) → (Prop → Prop → a → a → a)
→ (Prop → Prop → a → a → a) → (Prop → Prop → a → a → a)
→ Prop → a.

recProp fvar _ _ _ _ (Var s) = fvar s

recProp fvar fnot fy fo fimp prop = case of prop:

(No p) = fnot p (recn p)

(Y p1 p2) = fy p1 p2 (recn p1) (recn p2)

(O p1 p2) = fo p1 p2 (recn p1) (recn p2)

(Imp p1 p2) = fimp p1 p2 (recn p1) (recn p2).

where recn = recProp fvar fnot fy fo fimp.

¿DE QUÉ ES LA 'R'?

b) variablesR : Prop → [string].

variablesR = foldProp (\s → [s]) id f f f

where f = (\p1 p2 → p1 ++ p2) = (++)

variables :: Prop → [string]

variables p = eliminarRepetidos (variablesR p)

PODIAS USAR NUB, O MEJOR, UNION EN LUGAR DE ++

eliminarRepetidos :: [a] → [a]

eliminarRepetidos = rec id [] (\x xs r → if (elem x xs) then r else [x] ++ r)

¿rec?

FUNCIONABA IGUAL CON foldr

(x : r)

~~pertenece :: a → [a] → Bool~~

~~pertenece x = foldr (const False) (\~~

Asumo que elem y rec ya está definido en el preludio de Haskell

y tienen tipos elem :: a → [a] → Bool

rec :: b → (a → [a] → b → b) → [a] → b.

Valentín Fialkowski
N.o. = 81.

c) evaluar :: Valuación → Prop → Bool

~~evaluar val = recProp (\s → evaluar val s) (f val) (f val)~~
~~(f val) (f val)~~
~~where f val = \p r → ¬(val p)~~

evaluar val = foldprop (\s → val s) (\r → ¬r)
val not
(\r1 r2 → r1 && r2) (&&) not
(\r1 r2 → r1 || r2) (||)
(\r1 r2 → (¬r1) || r2).

d) está en FNN :: Prop → Bool

está en FNN = recProp . (const True)
. foux ¿QUÉ ES ESTE NOMBRE?
~~(_ r1 r2~~
. (_ _ r1 r2 → r1 && r2)
. (_ _ r1 r2 → r1 && r2)
. (_ _ _ _ → False)

~~whereaux~~
foux :: Prop → Bool → Bool
foux (Var _) _ = True
foux _ _ = False.

Valentín Fialkowski
N.º = 81.

(2)

a) demostrar:

$\forall t :: AIH\ a.\ \forall u :: AIH\ a.$ mismaEstructura $t\ u$ = mismaEstructura $u\ t$

Empiezo haciendo inducción sobre $t$.

· ~~Inducción~~

· El ppio. de inducción estructural me dice:

Sea $P(t) = \forall u :: AIH\ a.$ mismaEstructura $t\ u$
= mismaEstructura $u\ t$.

· si se cumple

$\forall n :: a.\ P(Hoja\ n)$ y

$\forall i :: AIH\ a.\ \forall d.\ AIH\ a.\ P(i) \wedge P(d) \rightarrow P(Bin\ i\ d)$.

entonces $\forall t :: AIH\ a.\ P(t)$.

· $P(Hoja\ n)$ (no escribo los $\forall$ pero están presentes)

: mismaEstructura $(Hoja\ n)\ u$ = mismaEstructura $u\ (Hoja\ n)$

(· Defino la macro ME = mismaEstructura).

. es Hoja $u$ = mismaEstructura $u\ (Hoja\ n)$ $\{M_0\}$.

Ahora utilizo extensionalidad sobre $u$: $(\exists m :: a)$.

· si $u :: AIH\ a$, entonces, o bien $u = Hoja\ m$

o bien $u = Bin\ l\ r$

$(\exists l :: AIH\ a.\ \exists r :: AIH\ a)$.

• caso $u$ = Hoja m.

• esHoja (Hoja m) = ME (Hoja m) (Hoja n)

$\downarrow$ {E0} $\qquad\qquad$ $\downarrow$ {M0}

True = esHoja (Hoja n)

$\downarrow$ {E0}

True = True ✓.

P(Hoja n) vale para $u$ = Hoja (m).

• caso $u$ = Bin l r

• esHoja (Bin l r) = ME (Bin l r) (Hoja n)

$\downarrow$ {E1} $\qquad\qquad$ $\downarrow$ {M1} {BETA}

False = not (EsHoja (Hoja n)) &&

ME l (izq (Hoja n)) &&

ME r (der (Hoja n)).

⇓

como, por {E0}: EsHoja (Hoja n) = True

entonces not (esHoja (Hoja n)) = False

reemplazando:

False = False && ME l (izq (Hoja n)) &&

ME l (der (Hoja n))

Por prop. de booleanos False && X = False para cualquier X :: Bool.

entonces:

Ejercicio c. Valentín Fiolkowski N.O = 81

False = False ✓.

Queda demostrado $P(\text{Hoja } n)$ para $u = (\text{Bin } l\ r)$.

• Ahora. $P(\text{Bin } i\ d) =$ (~~creído~~ presente el $\forall u :: \text{AlH } a$).

~~mostrar~~ • $ME\ (\text{Bin } i\ d)\ u = ME\ u\ (\text{Bin } i\ d)$.

[Aparte: tengo ~~la~~ la siguiente Hipótesis inductiva:

$P(i) \wedge P(d)$ : $(ME\ i\ u = ME\ u\ i \wedge$

$(\forall u :: \text{AlH } a)$ $ME\ d\ u = ME\ u\ d.)$]

Sigo con $P(\text{Bin } i\ d)$:

Reemplazando por $\{M\emptyset_1\}$ : $\{\text{BETA}\}$

[not (esHoja u) && ME i (izq u) && ME d (der u).

$= ME\ u\ (\text{Bin } i\ d)$.]

• Ahora uso extensionalidad ~~en~~ sobre $u$ de nuevo.

Si $u :: \text{AlH } a$, entonces

o bien ~~la~~ ~~[illegible]~~ $\exists m :: a.\ u = \text{Hoja } m$,

o bien $(\exists l :: \text{AlH } a.\ \exists r :: \text{AlH } a).\ u = (\text{Bin } l\ r)$.

y separo en casos para $u = \text{Hoja } m$ y $u = (\text{Bin } l\ r)$

• caso $u =$ Hoja $m$. Primero Analizo la parte izquierda de la igualdad:

$$\underbrace{\text{not (EsHoja (Hoja } m))}_{\text{False (por } \{E0\} \text{ y } \{\text{not}\})} \;\&\&\; \underbrace{\text{ME i izq (Hoja } m) \;\&\&\; \text{ME d der (Hoja } m)}_{\circledast}$$

$\therefore$ False && $\circledast$ = False. {&&}

P queda:

False = ME (Hoja m) (Bin i d)

False = EsHoja (Bin i d) {M0}

False = False ✓ {E1}.

Queda demostrado P(Bin i d) para $u =$ Hoja $m$. ✓

• caso $u =$ (Bin l r). Analizo la parte izq del '=' (en P)

not (EsHoja (Bin l r)) && ~~[illegible]~~

ME i izq (Bin l r) && ME d der (Bin l r).

reemplazo not (EsHoja (Bin l r)) por True (por {E2} y not).

True && ME i izq (Bin l r) && ME d der (Bin l r).

(por prop. de booleanos)

ME i izq (Bin l r) && ME d der (Bin l r)

Ahora, por {I} y por {D}

Queda: (ME i l) && (ME d r) = ME (Bin l r) (Bin i d)

Valentín Frolkowski
N.O. = 81
Ejercicio 2.

Ahora analizo la parte derecha de la igualdad.

ME (Bon l r) (Bon i d):

not EsHoja (Bon i d) && ME l (izq (Bon i d))
&& ME r (der (Bon i d)).

{HI}
{BOTH}

(procedimiento similar a la parte izquierda de la igualdad):

la propiedad queda: (Junto ahora los dos lados de la igualdad)
(P)

~~ME l i && ME r d.~~

FALTAN PASOS

ME i l && ME d r = ME l i && ME r d.

Ahora podemos aplicar la hipótesis inductiva P(i) ∧ P(d).

~~Como P(i): ME l ∀u :: AlHa ME i u = ME~~

como P(i) ∧ P(d): ∀u :: AlHa. (ME i u = ME u i ∧ ME d u = ME u d)

en particular: (ME i l = ME l i) ∧ (ME d r = ME r d)

y luego: ME i l && ME d r = ME l i && ME r d

↔ ME i l = ME l i ∧ ME d r = ME r d.

lo considero una propiedad de booleanos conocida.

. ~~Aplicando~~ lo cual vale por H.I. ✓

cuando demostrado P(Bon i d) para u = (Bon l r)

Por lo tanto. vale ∀t :: AlHa. P(t).

NO HACE FALTA, HACIENDO LOS REEMPLAZOS YA TE QUEDA LO QUE QUERÉS

Valentín Fidkowski

N.O. = 81.

Ejercicio 2) b)

b) ② $(\lambda x.\ x\ (\lambda x.\ succ(x)))\ (\lambda x.\ x)$.

Aplicación

⑥ $(\lambda x.\ x\ (\lambda x.\ succ(x)))$

Abs

⑤ $x\ (\lambda x.\ succ(x))$

Ap.

① $x$

④ $\lambda x.\ succ(x)$

Abs.

③. $succ(x)$

succ.

②. $x$

⑦ $(\lambda x.\ x)$

Abs.

⑨ $x$

ups. me olvidé de ese terminado. (me di cuenta cuando terminé ⑥) (lo llamo ⑨).

① . $W(x) \rightsquigarrow x : X_1 \vdash x : X_1$ $\quad X_1$: incógnita.

② . $W(x) \rightsquigarrow x : X_2 \vdash x : X_2$ $\quad X_2$: incógnita.

③ ~~W(succ~~

$$\frac{② . W(x) \rightsquigarrow x : X_2 \vdash x : X_2 \qquad s = mgu(X_2 \stackrel{?}{=} Nat)\ \ s = \{X_2 := Nat\}.}{W(succ(x)) \rightsquigarrow x : Nat \vdash succ(x) : Nat.}$$

④

$$\frac{③\ W(succ(x)) \rightsquigarrow x : Nat \vdash succ(x) : Nat}{W(\lambda x.\ succ(x)) \rightsquigarrow \emptyset \vdash \lambda x : Nat.\ succ(x) : Nat \to Nat}$$

⑤.

①: $w(x) \rightsquigarrow x : X_1 \vdash x : X_1$

②: $w(\lambda x.\, succ(x)) \rightsquigarrow$

$\emptyset \vdash \lambda x : Nat.\, succ(x) : Nat \to Nat.$

$S = mgu($

$\{X_1 \overset{?}{=} (Nat \to Nat) \to X_5\}$

$S = \{X_1 := (Nat \to Nat) \to X_5\}.$

---

$w(x\,(\lambda x.\, succ(x))) \rightsquigarrow x : (Nat \to Nat) \to X_5$

$\vdash x\,(\lambda x : Nat.\, succ(x)) : X_5$

⑥

③ $w(x\, \lambda x.\, succ(x)) \rightsquigarrow$

$\Gamma_1(x) = Nat \to Nat \to X_5$

$x : (Nat \to Nat) \to X_5 \vdash x\,(\lambda x : Nat.\, succ(x)) : X_5$

$\Gamma_1 =$

---

$w(\lambda x.\, x\,(\lambda x.\, succ(x))) \rightsquigarrow \emptyset \vdash (\lambda x : (Nat \to Nat) \to X_5.$

$x\,(\lambda x : Nat.\, suc(x)))$

$: \cancel{Nat \to Nat \to X_5}$

$: ((Nat \to Nat) \to X_5) \to X_5$

⑦

⑧ $w(x) \rightsquigarrow x : X_9 \vdash x : X_9.$ ✓

⑦.

⑧. $w(x) \rightsquigarrow x : X_9 \vdash x : X_9.$

---

$w(\lambda x.\, x) \rightsquigarrow \emptyset \vdash (\lambda x : X_9 .\, x) : X_9 \to X_9.$ ✓

⑧.

⑥ $w(\lambda x.\, x(\lambda x.\, succ(x)))$ $\rightsquigarrow$

$\emptyset \vdash (\lambda x{:}\, Nat \to Nat \to x_5 .\; x\ (\lambda x{:}\, Nat.\, succ(x)))$

$: ((Nat \to Nat) \to x_5) \to x_5$

⑦ $w(\lambda x.\, x)$

$\rightsquigarrow \emptyset \vdash (\lambda x{:}\, x_9 .\, x) : x_9 \to x_9$

$S = mgu(\{(Nat \to Nat) \to x_5) \to x_5 \overset{?}{=} (x_9 \to x_9) \to x_8\})$

$x_8$: incógnita.

---

$w((\lambda x.\, x(\lambda x.\, succ(x)))\ (\lambda x.\, x))$ $\rightsquigarrow$ $\emptyset \vdash$

⑥ ⑦

$((\lambda x{:}\, Nat \to Nat \to x_5 .\; x\ (\lambda x{:}\, Nat.\, succ(x)))\ (\lambda x{:}\, Nat \to Nat.\, x))$

$: x_5 \to x_5$

con $S = \{ x_9 := Nat,\ x_8 := x_5 \to x_5 \}$.

¡OJO CON LOS PARÉNTESIS!

Este es el juicio de tipado usado para la expresión.

Ej

2) b)

b) II. ⑧ $\lambda x.$ if iszero $(x)$ then $x$ else $x$ zero

↓ Abs

⑦ if iszero $(x)$ then $x$ else $x$ zero

⑦ ↓ ↓ ↓

② iszero$(x)$ ③ $x$ ⑥ $x$ zero

↓ ↓ ↯ Abs

① $x$ ④ $x$ ⑤ zero.

Valentin Tridkovsky

N.o. = 81

Ejercicio 2)b).

① $W(x) \rightsquigarrow x : X_1 \vdash x : X_1$

② ~~$W(iszero(x)) \rightsquigarrow$~~

$$\frac{W(x) \rightsquigarrow x : X_1 \vdash x : X_1 \qquad S = mgu(X_1 \overset{?}{=} Nat) = \{X_1 := Nat\}}{W(iszero(x)) \rightsquigarrow x : Nat \vdash iszero(x) : Bool.}$$

③ $W(x) \rightsquigarrow x : X_3 \vdash x : X_3.$

④ $W(x) \rightsquigarrow x : X_4 \vdash x : X_4$

⑤. $W(zero) \rightsquigarrow \emptyset \vdash zero : Nat.$

⑥.

$$\frac{\begin{array}{l} ④\ W(x) \rightsquigarrow x : X_4 \vdash x : X_4 \\ W(zero) \rightsquigarrow \emptyset \vdash zero : Nat \end{array} \qquad \begin{array}{l} S = mgu\{X_4 \overset{?}{=} Nat \to X_6\} \\ = \{X_4 := Nat \to X_6\}. \end{array}}{W(x\ zero) \rightsquigarrow x : Nat \to X_6 \vdash x\ zero : X_6.}$$

⑦.

② $W(iszero(x)) \rightsquigarrow x : Nat \vdash iszero(x) : Bool$

③. $W(x) \rightsquigarrow x : X_3 \vdash x : X_3$

⑥ $W(x\ zero) \rightsquigarrow x : Nat \to X_6 \vdash x\ zero : X_6$

$S = mgu($
$\{Bool \overset{?}{=} Bool,\ X_3 \overset{?}{=} X_6\}$
$\cup \{Nat \overset{?}{=} X_3,\ \cancel{X_3 \overset{?}{=}}$
$X_3 \overset{?}{=} Nat \to X_6\}.$

$Nat \overset{?}{=} Nat \to X_6$

$$\frac{W(if\ \underbrace{iszero(x)}_{②}\ then\ \underbrace{x}_{③}\ else\ \underbrace{x\ zero}_{⑥}) \rightsquigarrow \text{falla}.}{}$$

- ~~Sa~~ falla porque el algoritmo para encontrar a S. (martelli - montanari) falla

- justificación.

$$mgu(\{Bool \overset{?}{=} Bool,\ x_3 \overset{?}{=} x_6,\ \cancel{Nat}\ x_3 \overset{?}{=} Nat,\ x_3 \overset{?}{=} Nat \to x_6\})$$

$$\xrightarrow{delete} x_3 \overset{?}{=} x_6,\ x_3 \overset{?}{=} Nat,\ x_3 \overset{?}{=} Nat \to x_6$$

$$\xrightarrow[x_3 := x_6]{elim} x_6 \overset{?}{=} Nat,\ x_6 \overset{?}{=} Nat \to x_6.$$

$$\xrightarrow[x_6 := Nat]{elim} Nat \overset{?}{=} Nat \to Nat \longrightarrow \text{falla}$$

por clash.

Valentín Fidlkowski

N.o. = 81.

Ejercicio 2)b).

Valentín Fiałkowski
N.O. = 81.

Ejercicio 3

3) a) reglas de tipado.

$$\frac{\Gamma \vdash M : \tau}{\Gamma \vdash \text{Hoja}(m) : \text{AIH}(\tau)} \text{ hojam}$$

$$\frac{\Gamma \vdash M_1 : \text{AIH}(\tau) \qquad \Gamma \vdash M_2 : \text{AIH}(\tau)}{\Gamma \vdash \text{Bon}(M_1, M_2) : \text{AIH}(\tau)} \text{ Bon}$$

$$\frac{\Gamma \vdash M : \text{AIH}(\tau) \qquad \overset{x:t}{\Gamma \vdash M_1 : \sigma} \qquad \Gamma, i : \text{AIH}(\tau), d : \text{AIH}(\tau) : M_2 : \sigma}{\Gamma \vdash \text{case } M \text{ of Hoja } x \rightsquigarrow M_1 ; \text{Bon}(i,d) \rightsquigarrow M_2 : \sigma}$$

b)

$V ::= \ldots \mid \text{Hoja}(V) \mid \text{Bon}(V, V)$

nuevas reglas de semantica operacional: (cómputo)

- $\{\text{case}_1\}$: ~~case(Hoja(v))~~

~~case M of~~

$\text{case}(\text{Hoja } v) \text{ of Hoja } x \rightsquigarrow M_1 ; \text{Bon}(i,d) \rightsquigarrow M_2$

$\rightarrow M_1 \{x := v\}$.

{case2}

case Bon $(V_1, V_2)$ of Hoja $x \overset{M_1}{\rightarrow}$ ; Bon $(i,d) \rightsquigarrow M_2$

$\rightarrow M_2\{i := V_1\}\{d := V_2\}$.

reglas de congruencia: si $M \rightarrow M'$, $M_1 \rightarrow M_1'$, $M_2 \rightarrow M_2'$

{hoja1} Hoja $(M) \rightarrow$ Hoja $(M')$

{Bon1} Bon $(M_1, M_2) \rightarrow$ Bon $(M_1', M_2)$.

{Bon2} Bon $(V, M_2) \rightarrow$ Bon $(V, M_2')$.

{Case3} case $M$ of Hoja $x \rightarrow M_1$ ; Bon $(i,d) \rightsquigarrow M_2$

$\rightarrow$ case $M'$ of Hoja $x \rightsquigarrow M_1$ ; Bon $(i,d) \rightsquigarrow M_2$.

e) case $[(\lambda n: Nat.\ Hoja(n))\ succ(zero)]$

of Hoja $x \rightsquigarrow succ(pred(x))$ ; Bon $(i,d) \rightsquigarrow zero$

Por {case3} tengo que reducir:

$(\lambda n: Nat.\ Hoja(n))\ succ(zero)$

$\xrightarrow{\beta}$ Hoja $(n)\{n := succ(zero)\} = Hoja(succ(zero))$

case Hoja $(succ(zero))$ of Hoja $x \rightsquigarrow succ(pred\ x)$ ;

Bon $(i,d) \rightsquigarrow zero$.

Valentín Falkowski

N.O. = 81.

Usando {zero, }

$\rightarrow$ succ (pred x)) } x := succ (zero) }

{Derecho ?}

= succ (pred (succ (zero)).

succ

$\rightarrow$ {pred} succ (zero) ✓.